Internationalization in Java

Freshman Level: Your Program Meets the World

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-15

Outline

1. Printing Numbers
2. Reading Numbers
3. Turkish Letters
4. Upper and Lower Case
5. A First Translation
6. Summary

Your program will run on other computers

- Your laptop, your friend's laptop, the lab, the teacher's grading script, a server abroad: each may have a **different language and region**.
- Java then prints and reads numbers differently, sorts and capitalizes differently, and may show Turkish letters as ?.
- This deck: the **first-week surprises** and the **simple fixes**.

Try it on your computer

Windows: Settings › Time & language › Language & region

macOS: System Settings › General › Language & Region

Look at *Regional format* / *Region*: this is what Java uses.

On these slides `LANG=tr_TR.UTF-8` in front of a command plays a Turkish computer (macOS and Linux; on **Windows** change the setting above).

Printing Numbers

Same program, two computers

codeoutput/Receipt/Receipt.java — Java

```java
public class Receipt {
    public static void main(String[] args) {
        double price = 1249.9;
        int count = 3;
        System.out.printf("Price: %.2f%n",
      price);
        System.out.printf("Total: %,.2f%n",
      price * count);
        System.out.println("Plain: " + price *
      count);    // + never localizes
    }
}
```

`printf` and `String.format` use the computer's locale: `1249,90` with a **comma** in Turkish, and `,` groups thousands with a **point**. Plain `+` always prints Java's own way (`3749.7000000000003`).

Same program, two computers

codeoutput/Receipt/Receipt.java — Java

```java
public class Receipt {
    public static void main(String[] args) {
        double price = 1249.9;
        int count = 3;
        System.out.printf("Price: %.2f%n",
      price);
        System.out.printf("Total: %,.2f%n",
      price * count);
        System.out.println("Plain: " + price *
      count);    // + never localizes
    }
}
```

Terminal — codeoutput/Receipt.txt

```
$ java Receipt.java
Price: 1249.90
Total: 3,749.70
Plain: 3749.7000000000003
$ LANG=tr_TR.UTF-8 java Receipt.java
Price: 1249,90
Total: 3.749,70
Plain: 3749.7000000000003
```

`printf` and `String.format` use the computer's locale: `1249,90` with a **comma** in Turkish, and `,` groups thousands with a **point**. Plain `+` always prints Java's own way (`3749.7000000000003`).

Choose on purpose

codeoutput/ReceiptFixed/Receipt.java — Java

```java
import java.util.Locale;

public class Receipt {
    public static void main(String[] args) {
        double price = 1249.9;
        Locale tr = Locale.of("tr", "TR");
        System.out.printf(Locale.US, "for a file:   %.2f%n", price);   // always a point
        System.out.printf(tr, "for a person: %,.2f TL%n", price);      // Turkish style
        System.out.printf("default:      %,.2f%n", price);             // the computer decides
    }
}
```

- Output for **programs** (files, CSV, a grading script): give `Locale.US` or `Locale.ROOT`.
- Output for **people**: the user's locale, or a locale you choose on purpose.

Choose on purpose

codeoutput/ReceiptFixed/Receipt.java — Java

```java
import java.util.Locale;

public class Receipt {
    public static void main(String[] args) {
        double price = 1249.9;
        Locale tr = Locale.of("tr", "TR");
        System.out.printf(Locale.US, "for a file:   %.2f%n", price);   // always a point
        System.out.printf(tr, "for a person: %,.2f TL%n", price);      // Turkish style
        System.out.printf("default:      %,.2f%n", price);             // the computer decides
    }
}
```

Terminal — codeoutput/ReceiptFixed.txt

```
$ LANG=tr_TR.UTF-8 java Receipt.java
for a file:   1249.90
for a person: 1.249,90 TL
default:      1.249,90
```

- Output for **programs** (files, CSV, a grading script): give `Locale.US` or `Locale.ROOT`.
- Output for **people**: the user's locale, or a locale you choose on purpose.

Reading Numbers

Scanner reads the local way, too

codeoutput/Average/Average.java **Java**

```java
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Two numbers: ");
        double a = in.nextDouble();
        double b = in.nextDouble();
        System.out.println("Average: " + (a + b) / 2);
    }
}
```

On a Turkish computer `nextDouble()` wants `3,5`; `3.5` gives `InputMismatchException`. The program is right, the **input format** is wrong for that locale.

Scanner reads the local way, too

codeoutput/Average/Average.java — Java

```java
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Two numbers: ");
        double a = in.nextDouble();
        double b = in.nextDouble();
        System.out.println("Average: " + (a + b) / 2);
    }
}
```

Terminal — codeoutput/Average.txt

```
$ echo "3.5 4.5" | java Average.java
Two numbers: Average: 4.0
$ echo "3.5 4.5" | LANG=tr_TR.UTF-8 java Average.java 2>&1 | head -3
Two numbers: Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
        at java.base/java.util.Scanner.next(Scanner.java:1602)
$ echo "3,5 4,5" | LANG=tr_TR.UTF-8 java Average.java
Two numbers: Average: 4.0
```

On a Turkish computer `nextDouble()` wants `3,5`; `3.5` gives `InputMismatchException`. The program is right, the **input format** is wrong for that locale.

Fixing the input format

codeoutput/AverageFixed/Average.java — Java

```java
import java.util.Locale;
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in).useLocale(Locale.ROOT);  // '.' always
        System.out.print("Two numbers: ");
        double a = in.nextDouble();
        double b = in.nextDouble();
        System.out.println("Average: " + (a + b) / 2);
    }
}
```

`useLocale(Locale.ROOT)`: input with a decimal **point** on every computer. Use it for test data and files; keep the default when people type numbers the way they are used to.

Fixing the input format

codeoutput/AverageFixed/Average.java — Java

```java
import java.util.Locale;
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in).useLocale(Locale.ROOT);  // '.' always
        System.out.print("Two numbers: ");
        double a = in.nextDouble();
        double b = in.nextDouble();
        System.out.println("Average: " + (a + b) / 2);
    }
}
```

Terminal — codeoutput/AverageFixed.txt

```
$ echo "3.5 4.5" | LANG=tr_TR.UTF-8 java Average.
      java
Two numbers: Average: 4.0
```

`useLocale(Locale.ROOT)`: input with a decimal **point** on every computer. Use it for test data and files; keep the default when people type numbers the way they are used to.

parseDouble knows only the point

codeoutput/ParseError/Grade.java **Java**

```java
public class Grade {
    public static void main(String[] args) {
        double g = Double.parseDouble(args[0]);
      // Java syntax: point only
        System.out.println(g >= 60 ? "passed" : "
      failed");
    }
}
```

`Double.parseDouble` and `Integer.parseInt` never look at the locale: `"85,5"` is always an error. To accept `85,5`, use `NumberFormat.getInstance(locale).parse(s)` (next decks).

parseDouble knows only the point

codeoutput/ParseError/Grade.java — Java

```java
public class Grade {
    public static void main(String[] args) {
        double g = Double.parseDouble(args[0]);
      // Java syntax: point only
        System.out.println(g >= 60 ? "passed" : "
      failed");
    }
}
```

Terminal — codeoutput/ParseError.txt

```
$ java Grade.java 85.5
passed
$ java Grade.java 85,5 2>&1 | head -2
Exception in thread "main" java.lang.NumberFormatException:
        For input string: "85,5"
         at java.base/jdk.internal.math.FloatingDecimal.
        readJavaFormatString(FloatingDecimal.java:2054)
```

`Double.parseDouble` and `Integer.parseInt` never look at the locale: `"85,5"` is always an error. To accept `85,5`, use `NumberFormat.getInstance(locale).parse(s)` (next decks).

Turkish Letters

Where do the ? come from?

codeoutput/Letters/Letters.java **Java**

```java
public class Letters {
    public static void main(String[]
      args) {
        System.out.println("Çağrı Şükr
      ü Ilgın, İzmir");
    }
}
```

The `String` is fine; the **bytes** sent to the console depend on `stdout.encoding`. US-ASCII has no ç: ?. UTF-8 uses 2 bytes for Ç (c3 87), ISO-8859-9 one (c7): the console must expect the same encoding.

Where do the ? come from?

codeoutput/Letters/Letters.java — Java

```
1 public class Letters {
2     public static void main(String[]
        args) {
3         System.out.println("Çağrı Şükr
        ü Ilgın, İzmir");
4     }
5 }
```

Terminal — codeoutput/Letters.txt

```
$ java Letters.java
Çağrı Şükrü Ilgın, İzmir
$ java -Dstdout.encoding=US-ASCII Letters.java
?a?r? ??kr? Ilg?n, ?zmir
$ java Letters.java | od -An -tx1 | head -1
 c3 87 61 c4 9f 72 c4 b1 20 c5 9e c3 bc 6b 72 c3
$ java -Dstdout.encoding=ISO-8859-9 Letters.java | od -An -tx1 | head -1
 c7 61 f0 72 fd 20 de fc 6b 72 fc 20 49 6c 67 fd
```

The `String` is fine; the **bytes** sent to the console depend on `stdout.encoding`. US-ASCII has no ç: ?. UTF-8 uses 2 bytes for Ç (c3 87), ISO-8859-9 one (c7): the console must expect the same encoding.

Turkish letters on Windows and macOS

Windows (Command Prompt)

```
chcp 65001
java Letters.java
rem if letters are still wrong:
java -Dstdout.encoding=UTF-8 Letters.java
```

macOS (Terminal)

```
java Letters.java
# Terminal uses UTF-8: nothing to do
locale          # shows LANG, LC_ALL, ...
```

- **Windows** Command Prompt uses an old *code page* (857 on Turkish Windows, 437 on English Windows); `chcp 65001` switches it to UTF-8 for this window. Windows Terminal and the IDE consoles are usually fine.
- Save your `.java` files as **UTF-8** (all editors can). Since Java 18 `javac` reads UTF-8 by default; for older Java use `javac -encoding UTF-8`.

Upper and Lower Case

The bug that only Turkish users see

codeoutput/Menu/Menu.java **Java**

```java
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        String cmd = new Scanner(System.in).next();
        String key = cmd.toUpperCase();            // locale of the
        computer!
        System.out.println("key = " + key);
        if (key.equals("QUIT")) System.out.println("Bye.");
        else System.out.println("Unknown command: " + cmd);
    }
}
```

In Turkish, upper case `i` is `İ`, so `"quit"` becomes `"QUİT"`, which is not `"QUIT"`. The program works in the lab and fails for the user.

The bug that only Turkish users see

codeoutput/Menu/Menu.java — Java

```java
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        String cmd = new Scanner(System.in).next();
        String key = cmd.toUpperCase();            // locale of the
        computer!
        System.out.println("key = " + key);
        if (key.equals("QUIT")) System.out.println("Bye.");
        else System.out.println("Unknown command: " + cmd);
    }
}
```

Terminal — codeoutput/Menu.txt

```
$ echo quit | java Menu.java
key = QUIT
Bye.
$ echo quit | LANG=tr_TR.UTF-8 java Menu.
    java
key = QUİT
Unknown command: quit
```

In Turkish, upper case `i` is `İ`, so `"quit"` becomes `"QUİT"`, which is not `"QUIT"`. The program works in the lab and fails for the user.

The fix: `Locale.ROOT`

codeoutput/MenuFixed/Menu.java — Java

```java
import java.util.Locale;
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        String cmd = new Scanner(System.in).next();
        String key = cmd.toUpperCase(Locale.ROOT);   // same on every computer
        System.out.println("key = " + key);
        if (key.equals("QUIT")) System.out.println("Bye.");
        else System.out.println("Unknown command: " + cmd);
    }
}
```

Commands, keywords, file extensions: `toUpperCase(Locale.ROOT)` or `equalsIgnoreCase`. Names and sentences shown to people: `toUpperCase(userLocale)`.

The fix: `Locale.ROOT`

codeoutput/MenuFixed/Menu.java — Java

```java
import java.util.Locale;
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        String cmd = new Scanner(System.in).next();
        String key = cmd.toUpperCase(Locale.ROOT);   // same on every computer
        System.out.println("key = " + key);
        if (key.equals("QUIT")) System.out.println("Bye.");
        else System.out.println("Unknown command: " + cmd);
    }
}
```

Terminal — codeoutput/MenuFixed.txt

```
$ echo quit | LANG=tr_TR.UTF-8 java
      Menu.java
key = QUIT
Bye.
```

Commands, keywords, file extensions: `toUpperCase(Locale.ROOT)` or `equalsIgnoreCase`. Names and sentences shown to people: `toUpperCase(userLocale)`.

A First Translation

Texts in files, not in code

codeoutput/Greeter/Greeter.java — Java

```java
import java.util.ResourceBundle;
import java.util.Scanner;

public class Greeter {
    public static void main(String[] args) {
        ResourceBundle text = ResourceBundle.getBundle("Texts"); // default locale
        System.out.print(text.getString("ask") + " ");
        String name = new Scanner(System.in).nextLine();
        System.out.println(text.getString("hello") + ", " + name + "!");
    }
}
```

codeoutput/Greeter/Texts.properties — properties

```properties
# English: the base file, used when nothing better exists
ask=What is your name?
hello=Hello
```

codeoutput/Greeter/Texts_tr.properties — properties

```properties
# Turkish
ask=Adınız nedir?
hello=Merhaba
```

`getBundle("Texts")` looks for `Texts_tr.properties` on a Turkish computer and `Texts.properties` otherwise. Files go next to the classes (on the class path).

One program, two languages

Terminal codeoutput/Greeter.txt

```
$ ls
Greeter.java
Texts.properties
Texts_tr.properties
$ echo Ayşe | java Greeter.java
What is your name? Hello, Ayşe!
$ echo Ayşe | LANG=tr_TR.UTF-8 java Greeter.java
Adınız nedir? Merhaba, Ayşe!
$ echo Ayşe | java -Duser.language=tr Greeter.java
Adınız nedir? Merhaba, Ayşe!
```

No `if (turkish)` in the code: adding French means adding `Texts_fr.properties`.

First errors with bundles

Terminal codeoutput/Missing.txt

```
$ mkdir other
$ cd other
$ echo Ali | java ../Greeter.java 2>&1 | head -2
Exception in thread "main" java.util.MissingResourceException: Can't find bundle for base name Texts, locale en_US
        at java.base/java.util.ResourceBundle.throwMissingResourceException(ResourceBundle.java:2059)
$ cd ..
$ grep Merhaba Texts_tr.properties
helo=Merhaba
$ echo Ali | java -Duser.language=tr Greeter.java
Adınız nedir? Hello, Ali!
$ grep getString Greeter.java
        System.out.print(text.getString("ask") + " ");
        System.out.println(text.getString("bye") + ", " + name + "!");
$ echo Ali | java -Duser.language=tr Greeter.java 2>&1 | head -2
Adınız nedir? Exception in thread "main" java.util.MissingResourceException: Can't find resource for bundle java.util.PropertyResourceBundle,
        key bye
        at java.base/java.util.ResourceBundle.getObject(ResourceBundle.java:567)
```

1: run from another folder: the `.properties` files are not on the class path, `MissingResourceException` for the *bundle*. 2: a typo in the Turkish file: the key is taken from the base file (English). 3: a key that exists nowhere: `MissingResourceException` for the *key*.

Summary

Cheat sheet

problem	fix
`3,14` instead of `3.14`	`printf(Locale.US, "%.2f", x)` for programs
`InputMismatchException`	`new Scanner(System.in).useLocale(Locale.ROOT)`
`"85,5"` gives `NumberFormatException`	`NumberFormat.getInstance(locale).parse(s)`
`?` instead of `ç`	**Windows**: `chcp 65001`; files saved as UTF-8
`"quit".toUpperCase()` is `"QUİT"`	`toUpperCase(Locale.ROOT)`
texts in two languages	`ResourceBundle.getBundle("Texts")` + `.properties` files
`MissingResourceException`	files on the class path? key spelled right?

Do and do not

Do

- test your program once with a Turkish locale
- use `Locale.ROOT` for keywords and data files
- save sources as UTF-8
- keep user texts in `.properties` files

Do not

- assume everyone writes `3.5`
- compare commands after `toUpperCase()` without a locale
- glue sentences: `"Hello, " + name + "!"` cannot be translated well
- ignore `?` in the output: the data may already be lost

Exercises

1. Change your regional format to Turkish, run `Receipt`, then change it back.
2. Write a program that reads prices in Turkish style (`12,50`) and prints the total in US style.
3. Find the bug: `if (ext.toLowerCase().equals("ini"))` for the file `SETUP.INI`. On which computers does it fail? Why?
4. Add `Texts_de.properties` to the greeter and run it with `-Duser.language=de`.
5. Make the greeter print the date in the user's language (look at `DateTimeFormatter.ofLocalizedDate`).

Questions?